

python重要数据结构复习

1. 元组

创建元组

空元组

带初始值的元组

访问元组中的元素

修改元组中的元素

元组解包

嵌套元组

单元素元组

元组方法

例子

2. 列表

a. 创建列表

i.  空列表

ii.  带初始值的列表

b. 访问列表中的元素

c. 修改列表中的元素

d. 添加元素

e. 删除元素

f. 列表切片

g. 列表遍历

h. 列表方法

i. 例子

3. 字典

a. 创建字典

i.  空字典

ii.  带初始值的字典

b. 访问字典中的值

- c. 修改字典中的值
- d. 添加新的键值对
- e. 删除键值对
- f. 检查键是否存在
- g. 遍历字典
- h. 字典方法
- i. 示例字典

1. 元组

在 Python 中，元组（tuple）是一种**有序的、不可变**的集合。与列表类似，元组也可以包含任意类型的元素，但一旦创建就不能修改。元组使用圆括号 `()` 来定义，元素之间用逗号分隔。

创建元组

空元组

```
▼ Python |
1 empty_tuple = ()
```

带初始值的元组

```
▼ Python |
1 fruits = ("apple", "banana", "cherry")
2 numbers = (1, 2, 3, 4, 5)
3 mixed = (1, "apple", 3.14, True)
```

访问元组中的元素

可以通过索引来访问元组中的元素，索引从 0 开始：

```
▼ Python |
1 first_fruit = fruits[0] # "apple"
2 second_number = numbers[1] # 2
```

修改元组中的元素

元组是不可变的，所以不能直接修改元组中的元素。如果需要修改，可以将元组转换为列表，进行修改后再转换回元组：

```
1 temp_list = list(fruits)
2 temp_list[1] = "blueberry"
3 fruits = tuple(temp_list)
```

元组解包

元组解包允许你将元组的元素分别赋值给多个变量：

```
1 fruit1, fruit2, fruit3 = fruits
2 print(fruit1) # "apple"
3 print(fruit2) # "banana"
4 print(fruit3) # "cherry"
```

嵌套元组

元组可以包含其他元组作为元素：

```
1 nested_tuple = (1, (2, 3), 4)
2 inner_tuple = nested_tuple[1] # (2, 3)
```

单元素元组

定义单元素元组时，需要在元素后添加一个逗号，否则会被认为是普通括号运算：

```
1 single_element_tuple = (5,)
2 not_a_tuple = (5) # 这是一个整数，不是元组
```

元组方法

尽管元组是不可变的，但它有一些方法和操作：

- `count(x)`：返回元组中值为 `x` 的元素个数。
- `index(x)`：返回元组中第一个值为 `x` 的元素的索引。

例子

```
1  # 创建元组
2  coordinates = (10.0, 20.0, 30.0)
3  # 访问元素
4  x = coordinates[0]
5  y = coordinates[1]
6  # 元组解包
7  latitude, longitude, altitude = coordinates
8  # 嵌套元组
9  nested = ("point", (10.0, 20.0), "end")
10 label, position, end_label = nested
11 # 单元素元组
12 single_element = (5,)
```

2. 列表

在 Python 中，列表（list）是一种有序的、可变的集合，可以包含任意类型的元素。列表使用方括号 `[]` 来定义，元素之间用逗号分隔。

a. 创建列表

i. 空列表

```
1  empty_list = []
```

ii. 带初始值的列表

```
1 fruits = ["apple", "banana", "cherry"]
2 numbers = [1, 2, 3, 4, 5]
3 mixed = [1, "apple", 3.14, True]
```

b. 访问列表中的元素

可以通过索引来访问列表中的元素，索引从 0 开始：

```
1 first_fruit = fruits[0] # "apple"
2 second_number = numbers[1] # 2
```

c. 修改列表中的元素

可以通过索引来修改列表中的元素：

```
1 fruits[1] = "blueberry" # 修改第二个元素为 "blueberry"
```

d. 添加元素

可以使用 `append()` 方法在列表末尾添加元素：

```
1 fruits.append("orange") # 在列表末尾添加 "orange"
```

可以使用 `insert()` 方法在指定位置插入元素：

```
1 fruits.insert(1, "kiwi") # 在索引 1 位置插入 "kiwi"
```

e. 删除元素

可以使用 `remove()` 方法删除指定值的元素：

```
1 fruits.remove("banana") # 删除值为 "banana" 的元素
```

可以使用 `pop()` 方法删除指定索引的元素（不指定索引时删除最后一个元素）：

```
1 fruits.pop(1) # 删除索引 1 处的元素
2 last_fruit = fruits.pop() # 删除并返回最后一个元素
```

f. 列表切片

可以使用切片操作访问列表的一个子集：

```
1 subset = numbers[1:4] # 获取索引 1 到 3 的元素
```

g. 列表遍历

可以使用 `for` 循环遍历列表中的每个元素：

```
1 for fruit in fruits:
2     print(fruit)
```

h. 列表方法

Python 提供了一些常用的列表方法：

- `append(x)`：在列表末尾添加一个元素 `x`。
- `extend(iterable)`：使用一个可迭代对象扩展列表。
- `insert(i, x)`：在指定位置 `i` 插入元素 `x`。
- `remove(x)`：删除列表中第一个值为 `x` 的元素。
- `pop([i])`：删除并返回指定位置 `i` 处的元素，如果没有指定位置，则删除并返回最后一个元素。
- `clear()`：移除列表中的所有元素。

- `index(x[, start[, end]])` : 返回列表中第一个值为 `x` 的元素的索引, 如果没有找到则抛出 `ValueError`。
- `count(x)` : 返回列表中值为 `x` 的元素的个数。
- `sort(key=None, reverse=False)` : 对列表中的元素进行排序。
- `reverse()` : 反转列表中的元素。
- `copy()` : 返回列表的浅拷贝。

i. 例子

```
1  # 创建列表
2  animals = ["cat", "dog", "rabbit", "hamster"]
3  # 访问和修改元素
4  print(animals[1]) # 输出: dog
5  animals[1] = "parrot"
6  # 添加和删除元素
7  animals.append("fish")
8  animals.insert(2, "turtle")
9  animals.remove("cat")
10 removed_animal = animals.pop(3)
11 # 列表切片
12 subset = animals[1:3]
13 # 列表遍历
14 for animal in animals:
15     print(animal)
```

3. 字典

在 Python 中, 字典 (dictionary) 是一种可变的、无序的键值对集合。每个键 (key) 和值 (value) 之间通过冒号 (`:`) 分隔, 每对键值之间使用逗号 (`,`) 分隔, 并且整个集合用花括号 (`{}`) 包裹。下面是对 Python 字典结构的详细介绍:

a. 创建字典

i. 空字典

```
1 empty_dict = {}
```

ii. 带初始值的字典

```
1 person = {  
2     "name": "Alice",  
3     "age": 30,  
4     "city": "New York"  
5 }
```

b. 访问字典中的值

可以通过键来访问对应的值：

```
1 name = person["name"] # "Alice"
```

c. 修改字典中的值

可以直接通过键来修改对应的值：

```
1 person["age"] = 31
```

d. 添加新的键值对

可以直接通过键来添加新的键值对：

```
1 person["email"] = "alice@example.com"
```

e. 删除键值对

可以使用 `del` 关键字或 `pop` 方法来删除键值对：

```
1 del person["city"]
2 email = person.pop("email")
```

f. 检查键是否存在

可以使用 `in` 关键字来检查键是否在字典中：

```
1 if "name" in person:
2     print("Name is in the dictionary")
```

g. 遍历字典

可以使用 `items()` 方法来遍历字典中的键值对：

```
1 for key, value in person.items():
2     print(f"{key}: {value}")
```

h. 字典方法

Python 提供了一些常用的字典方法：

- `get(key, default)`：返回指定键的值，如果键不在字典中，返回默认值。
- `keys()`：返回所有键的视图。
- `values()`：返回所有值的视图。
- `items()`：返回所有键值对的视图。
- `update([other])`：用其他字典中的键值对更新字典。

i. 示例字典

结合代码片段，假设 `self.json` 的结构可能如下：

```
1 self.json = {  
2     "strategies": [  
3         {  
4             "config": {  
5                 "vehicle_param_value": {  
6                     "rear_axle_to_head": 4.2,  
7                     "rear_axle_to_back": 1.1,  
8                     "half_width": 1.0  
9                 }  
10            }  
11        }  
12    ]  
13 }
```

在这个结构中：

- `self.json` 是一个字典，包含一个键 `"strategies"`，其值是一个列表。
- 列表的第一个元素是一个字典，包含一个键 `"config"`，其值也是一个字典。
- `"config"` 字典包含键 `"vehicle_param_value"`，其值又是一个字典，存储具体的车辆参数。